



GOBERNAR



SESIÓN 15 · UNIDAD IV · ECOSISTEMA

¿QUIÉN MANDA CUANDO NO HAY DUEÑO?

Gobernar un sistema sin jefe: firmas múltiples, propuestas, votos, un plazo de gracia y ejecución en el código. Y quién eres tú en una cadena.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

Del contrato con un solo dueño a la organización sin dueño. Y una conversación difícil: qué significa tu identidad cuando vive en una cadena pública.

A	DAOS Y GOBERNANZA Multisig, Safe, Governor + Timelock, quórum, delegación, modelos de voto y el caso Beanstalk.	~60 min
B	IDENTIDAD DESCENTRALIZADA DID, credenciales verificables, SBT y el choque con el habeas data.	~20 min
C	LABORATORIO 15 Una DAO con timelock, el ciclo completo con tiempo simulado, y un multisig 2 de 3.	~70 min
D	EL ENSAYO INDIVIDUAL (RA7) Se plantea hoy; se entrega en la Sesión 17. 8 % de la nota final.	~20 min



AL SALIR DE AQUÍ

Implementar un mecanismo de gobernanza on-chain con timelock y evaluar críticamente los modelos de identidad descentralizada.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

- 01** ¿Cuándo un multisig es la respuesta correcta y cuándo hace falta votación?
- 02** ¿Cómo pasa una propuesta de una idea a una ejecución automática, con plazo de gracia?
- 03** ¿Qué falla en «un token, un voto», y cómo se compró una votación en 2022?
- 04** ¿Qué tensión hay entre una identidad on-chain y el derecho a suprimir tus datos?

RA2 · arquitecturas de gobernanza · RA3 · Governor + Timelock · RA7 · identidad, ética y habeas data.



SIGLAS Y TÉRMINOS DE HOY

DAO*Decentralized Autonomous Organization*

Organización cuyas reglas viven en contratos y cuyas decisiones toman sus miembros, sin jefe único.

GOVERNOR*contrato de gobierno*

Contrato de OpenZeppelin que recibe propuestas, cuenta votos y manda ejecutar lo aprobado.

QUÓRUM*quorum*

Mínimo de poder de voto que debe participar para que una votación valga.

DID*Decentralized Identifier*

Identificador que controla su dueño con una clave, no una plataforma. W3C, 2022.

MULTISIG*multifirma · m de n*

Cuenta que exige m firmas de n posibles para actuar. Safe es el estándar de facto.

TIMELOCK*TimelockController*

Plazo obligatorio entre aprobar y ejecutar: la salida de emergencia de la minoría.

DELEGACIÓN*delegate()*

Activar tu poder de voto, en ti o en otra cuenta. Sin delegar, tus tokens votan cero.

SBT*Soulbound Token*

Token no transferible atado a una cuenta, para credenciales y reputación.



✓ APROXIMADAMENTE 60 MINUTOS

DAOS Y GOBERNANZA

Gobernar sin jefe suena utópico. En la práctica es un espectro: desde un multisig de tres personas hasta miles votando on-chain, cada punto con sus grietas.



EL PROBLEMA DEL DUEÑO ÚNICO

Desde la Sesión 6 casi todos los contratos del curso tienen un dueño: una sola dirección con permisos especiales (onlyOwner, soloEmisor). Es cómodo. Y es un punto único de falla.

SI ESA CLAVE...	LO QUE PASA CON EL CONTRATO
se pierde (se daña el disco, se olvida la frase)	Nadie vuelve a poder pausar, emitir o cambiar nada. Para siempre.
se la roban	El ladrón tiene TODOS los poderes del dueño: vaciar, pausar, cambiar reglas.
pertenece a alguien que se va del equipo	El proyecto depende de la buena voluntad de una persona que ya no está.



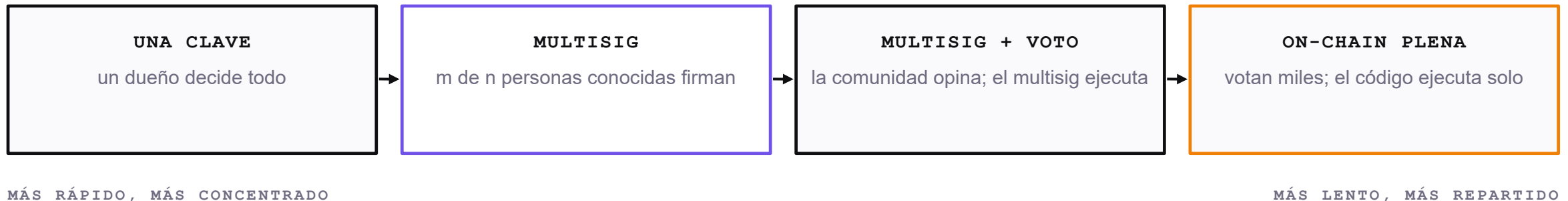
LA PREGUNTA QUE ORGANIZA EL BLOQUE

¿Cómo se reparte el poder sobre un contrato entre varias personas, de modo que ninguna sola pueda perderlo, robarlo o abusar de él? Hay dos respuestas: firmas múltiples y votación. Hoy vemos las dos.



UN ESPECTRO, NO UNA COSA

«DAO» se usa para cosas muy distintas. Ordenémoslas por cuánta gente decide y cuánto hace el código solo:



EL ERROR DE CONCEPTO TÍPICO

«Si es una DAO, está descentralizada». No: muchas «DAOs» son, en la práctica, un multisig de cinco fundadores con una votación consultiva encima.

El consejo que sorprende: casi siempre, al principio, lo correcto es la segunda casilla. La cuarta tiene sentido con una comunidad grande y real que gobernar, no antes.



M DE N: NINGUNA LLAVE SOLA ABRE LA CAJA

Una cuenta multifirma exige que m de n personas firmen para ejecutar una transacción. Lo que se elige al crearla son dos números, y cada combinación resiste cosas distintas:

ESQUEMA	SOBREVIVE A PERDER...	RESISTE QUE ROBEN...	COMENTARIO
1 de 2	1 clave	0 claves	Una llave de repuesto. Cualquiera de los dos puede vaciarla.
2 de 2	0 claves	1 clave	Si uno pierde su clave, los fondos quedan congelados.
2 de 3	1 clave	1 clave	El equilibrio típico de un equipo pequeño.
3 de 5	2 claves	2 claves	Tesorerías de proyectos; más lento de coordinar.

regla: sobrevive a perder $n - m$ claves · resiste que roben $m - 1$ claves

Error típico: un 2 de 3 donde las tres claves viven en el mismo portátil es, en la práctica, un 1 de 1.



SAFE: FIRMAR GRATIS, EJECUTAR UNA VEZ

01 **CREAR** En app.safe.global se eligen red, firmantes y umbral. Con 2 o más firmantes, el despliegue paga gas.

02 **PROPONER** Un firmante arma la transacción y la firma. La firma NO es una transacción: no cuesta gas.

03 **CONFIRMAR** El segundo firmante revisa y firma también, fuera de la cadena. Queda en la cola.

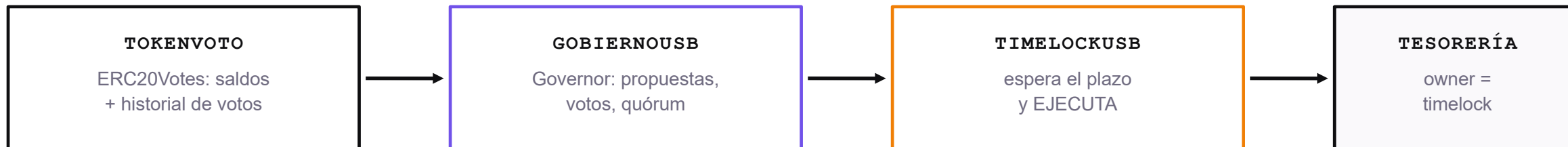
04 **EJECUTAR** Con el umbral reunido, cualquiera ejecuta: UNA transacción que lleva todas las firmas. Solo ella paga gas.

[] VERIFICADO HOY PARA EL LABORATORIO

Safe{Wallet} soporta Sepolia (sí) y NO lista OP Sepolia. El laboratorio usa Sepolia. Y trae un multisig mínimo propio (MultisigUSB) para ver el mecanismo por dentro sin red.



TOKEN, GOVERNOR, TIMELOCK Y TESORERÍA



PIEZA	LE PREGUNTA A...	ERROR TÍPICO
Governor	al token: ¿cuántos votos tenía cada quien en el bloque de la propuesta?	Crear que cuenta el saldo de HOY: cuenta el de ese bloque.
Timelock	al Governor: ¿esta operación fue aprobada? Solo él puede agendar.	Dejar al que desplegó como administrador: un dueño escondido.
Tesorería	al que llama: ¿eres mi dueño?	Poner de dueño al Governor: quien llama al final es el TIMELOCK.

Las cuatro piezas del laboratorio. La flecha es el camino del poder: nadie se la puede saltar.

SIETE PASOS, CINCO ESTADOS

01	PROPONER · PENDING	Alguien presenta: «liberar 2 ETH de la tesorería al semillero». Queda fijado el bloque de conteo.
02	RETRASO	Un plazo antes de abrir la votación, para que todos se enteren (aquí: 1 bloque).
03	VOTAR · ACTIVE	A favor, en contra o abstención, con el peso de los votos delegados (aquí: 50 bloques).
04	CERRAR · SUCCEEDED	Si hubo quórum y ganaron los votos a favor. Si no: Defeated, y se acabó.
05	ENCOLAR · QUEUED	queue() agenda la operación en el timelock. Empieza a correr el plazo de gracia.
06	ESPERAR	Pasa el retardo mínimo (en segundos). Quien no esté de acuerdo puede irse a tiempo.
07	EJECUTAR · EXECUTED	Cualquiera llama execute(). El timelock hace exactamente lo aprobado.



UNA PROPUESTA REAL DEL LABORATORIO

Ana tiene 60 votos, Beto 30 y Carla 10 (100 en total, todos delegados). Quórum: 4 % = 4 votos. Ana propone en el bloque 100.

MOMENTO	BLOQUE / TIEMPO	QUÉ PASA	ESTADO
<code>propose()</code>	bloque 100	Se fija el snapshot: bloque 101 (retraso de 1).	Pending
<code>votación abierta</code>	102 a 151	Ana vota a favor (60); Beto en contra (30).	Active
<code>cierre</code>	bloque 152	Participaron $90 \geq 4$ (quórum) y $60 > 30$.	Succeeded
<code>queue()</code>	hora t	El timelock agenda la operación: ejecutable desde $t + 3\,600$ s.	Queued
<code>execute()</code> antes	$t + 10$ min	Revierte: <code>TimelockUnexpectedOperationState</code> .	Queued
<code>execute()</code>	$t + 1$ h	La tesorería envía 2 ETH a Carla.	Executed

Estos son los números de la prueba «★ proponer, votar, encolar, esperar y ejecutar». En la red local el tiempo se simula: `mine(51)` salta bloques; `time.increase(3601)` salta segundos.

DOS CONDICIONES DISTINTAS, LAS DOS OBLIGATORIAS

En el conteo simple de OpenZeppelin: el quórum se mide con los votos A FAVOR + ABSTENCIÓN; la mayoría, con A FAVOR frente a EN CONTRA. Mismo padrón de 100 votos, quórum 4:

ESCENARIO	A FAVOR	EN CONTRA	ABSTENCIÓN	RESULTADO
Solo Carla vota a favor	10	0	0	Pasa: $10 \geq 4$ y $10 > 0$
Una cuenta con 1 voto vota a favor	1	0	0	Derrotada: $1 < 4$, falta quórum aunque sea 100 % a favor
Ana en contra, Carla a favor	10	60	0	Derrotada: hay quórum, pero $10 < 60$
Beto se abstiene, Carla a favor	10	0	30	Pasa: $40 \geq 4$ y $10 > 0$

⚠ EL ERROR DE CONCEPTO TÍPICO

«Si nadie vota en contra, pasa». No: sin quórum, una propuesta 100 % a favor también es derrotada. Lo prueba «sin quórum, la propuesta es derrotada...».



SIN DELEGAR, TUS TOKENS VOTAN CERO

El detalle que confunde a todos: tener tokens de gobierno no basta. Hay que DELEGAR el voto —a uno mismo o a otro— para que cuente.

LO QUE MUESTRAN LAS PRUEBAS DE TEST/S15/GOBIERNO.TEST.JS

```
await token.acunar(ana, 10)           // balanceOf(ana) = 10
await token.getVotes(ana)              // → 0           ;sin delegar!
await token.connect(ana).delegate(ana)
await token.getVotes(ana)              // → 10
await token.connect(ana).delegate(beto)
await token.getVotes(beto)             // → 10          (los tokens siguen siendo de Ana)
```

POR QUÉ EXISTE

Registrar el poder de voto en cada bloque cuesta gas (los checkpoints). Exigir delegar hace que solo se registre quien quiere participar.

PARA QUÉ SIRVE

Ceder tu voz a alguien que sigue el tema, sin darle tus tokens: la democracia representativa, en un contrato.



EL VOTO SE CONGELA AL PROPONER

El Governor no pregunta cuántos votos tienes HOY: pregunta cuántos tenías en el bloque del snapshot. Consecuencia: comprar votos después de que se publica la propuesta no sirve de nada.

01

BLOQUE 100

Ana propone. El snapshot queda en el bloque 101.

02

BLOQUE 103

Un atacante consigue 1 000 votos: diez veces todo lo demás.

03

BLOQUE 104

Vota en contra. $\text{getVotes}(\text{atacante}, 101) = 0$: su voto pesa CERO.



POR QUÉ IMPORTA: EL PRÉSTAMO RELÁMPAGO

Un flash loan (Sesión 14) presta millones dentro de UNA transacción y exige devolverlos antes de que termine. No puede cambiar el saldo de un bloque que ya pasó. El snapshot es la defensa. Veremos qué pasó cuando no la había.



EL PLAZO DE GRACIA DE LA MINORÍA

Una propuesta aprobada no se ejecuta enseguida: se encola y espera un retardo mínimo. Ese plazo protege al que perdió la votación: si la mayoría aprobó algo que lo perjudica, alcanza a retirar sus fondos antes.

ROL DEL TIMELOCKCONTROLLER	QUIÉN LO TIENE EN LA DAO DEL LABORATORIO
PROPOSER · agenda operaciones	Solo el Governor. Nadie puede saltarse la votación.
CANCELLER · cancela lo agendado	Solo el Governor.
EXECUTOR · ejecuta lo vencido	Cualquiera (address(0)): no hace falta confiar en nadie para ejecutar.
ADMIN · reparte roles	Nadie. Quien desplegó RENUNCIA al terminar de configurar.



EL DUEÑO ESCONDIDO

Si el que desplegó conserva el rol ADMIN, puede darse a sí mismo PROPOSER y agendar lo que quiera sin votación. La prueba «★ el fundador renunció...» verifica que ya no puede.



«UN TOKEN, UN VOTO» Y SUS ALTERNATIVAS

MODELO	CÓMO CUENTA	A FAVOR	EL PROBLEMA
Un token, un voto	Poder proporcional a los tokens.	Simple; alinea voto y capital en riesgo.	Plutocracia: quien más tiene, más manda.
Cuadrático	n votos cuestan n^2 tokens.	Da peso a la mayoría frente a unas pocas ballenas.	Se rompe si una persona se divide en muchas cuentas (Sybil, Sesión 4).
Por convicción	El peso crece cuanto más tiempo se mantiene el voto.	Premia el compromiso sostenido, no la compra de último minuto.	Más difícil de entender y de auditar; decisiones lentas.



NINGUNO ES NEUTRAL

Cada regla de conteo reparte el poder de una forma distinta. Elegir el modelo ES una decisión política, no técnica. En el laboratorio usamos el primero porque es el que trae OpenZeppelin, no porque sea el mejor.



POR QUÉ FRENA A LA BALLENA, Y CÓMO SE ROMPE

votos = $\sqrt{\text{tokens gastados}}$ (1 voto cuesta 1 · 10 votos cuestan 100 · 100 votos cuestan 10 000)

QUIÉN	TOKENS	UN TOKEN, UN VOTO	CUADRÁTICO
Una ballena	10 000	10 000 votos	$\sqrt{10\,000} = 100$ votos
100 estudiantes con 100 cada uno	10 000	10 000 votos	$100 \times \sqrt{100} = 1\,000$ votos
La ballena partida en 100 cuentas de 100	10 000	10 000 votos	$100 \times \sqrt{100} = 1\,000$ votos

LO QUE LOGRA

Con el mismo capital, cien personas pesan diez veces más que una sola ballena. La intensidad cuenta, pero con rendimientos decrecientes.

CÓMO SE ROMPE

Última fila: si la ballena crea 100 cuentas, recupera los 1 000 votos. El cuadrático SOLO funciona si se sabe que cada cuenta es una persona distinta.



PLUTOCRACIA, APATÍA Y CAPTURA

Una cuenta que se hace en un minuto: suministro de 100 millones de tokens, quórum del 4 % (4 millones). Si solo vota el 5 % (5 millones)...

3 M

Con 3 millones de votos, una sola cuenta es el 60 % de lo emitido: decide sola.

4 M

Con 4 millones, alcanza el quórum SOLA, aunque nadie más vote.

95 %

de los votos posibles nunca aparecieron. La apatía concentra el poder real.

MAL	QUÉ ES
Plutocracia	Quien más tokens tiene, más manda. Es el diseño funcionando, no un error.
Apatía	La mayoría no vota nunca; el poder efectivo queda en pocas manos.
Captura	Una entidad acumula el poder de voto de forma legal y aprueba lo que quiere. No es un hackeo: por eso es difícil de remediar.



UNA VOTACIÓN COMPRADA POR SEGUNDOS

01 UN DÍA ANTES El atacante presenta la propuesta BIP-18: transferirse los fondos del protocolo. Y espera el retraso de ~24 h.

02 PRESTAR En UNA transacción pide préstamos relámpago: unos USD 1 000 millones en stablecoins de Aave, y más en otros protocolos.

03 COMPRAR VOTOS Convierte lo prestado en poder de voto de Beanstalk, que se medía en ese mismo momento.

04 APROBAR Y EJECUTAR Con más de 2/3 de los votos, la «ejecución de emergencia» corre BIP-18 en esa misma transacción.

05 DEVOLVER Paga los préstamos y se queda con la diferencia. Todo en un solo bloque.

~USD 181 M

drenados del protocolo

~USD 76 M

ganancia neta del atacante (24 830 WETH)



LAS DOS DEFENSAS DEL LABORATORIO DE HOY

SNAPSHOT EN UN BLOQUE PASADO

Si el poder de voto se hubiera medido en el bloque de la propuesta (un día antes), los tokens prestados en el bloque del ataque habrían pesado cero. Es exactamente lo que hace ERC20Votes con sus checkpoints.

TIMELOCK SIN ATAJS

La «ejecución de emergencia» permitía aprobar y ejecutar en la misma transacción. Con un timelock obligatorio entre aprobar y ejecutar, el préstamo relámpago —que dura una transacción— no alcanza a llegar a la ejecución.



LA LECCIÓN QUE SIRVE MÁS ALLÁ DE BEANSTALK

El código hizo exactamente lo que decía. No hubo un error de programación clásico: hubo un diseño de gobernanza que confundió «tener tokens ahora» con «tener derecho a decidir». Los atajos «de emergencia» son siempre el punto a auditar primero.



SIETE IDEAS SOBRE GOBERNAR SIN JEFE

01

«DAO» es un espectro; casi siempre, empezar por un multisig es lo correcto.

02

m de n sobrevive a perder $n - m$ claves y resiste que roben $m - 1$.

03

El ciclo: proponer → votar → encolar → esperar → ejecutar. Nadie ejecuta a mano.

04

Sin delegar, los tokens votan cero. Y cuenta el saldo del snapshot, no el de hoy.

05

Quórum (a favor + abstención) y mayoría (a favor > en contra) son dos pruebas distintas.

06

El timelock ejecuta y es dueño de la tesorería; quien desplegó renuncia al rol admin.

07

Ningún modelo de voto es neutral; el cuadrático exige saber quién es quién.



PREGUNTA DE CONTROL

Un equipo despliega TokenVoto, GobiernoUSB y TimelockUSB, pone la tesorería a nombre del Governor y no renuncia al rol admin. Una propuesta gana. ¿Qué pasa al ejecutarla, y qué riesgo quedó abierto?

RESPUESTA ESPERADA (MOSTRAR DESPUÉS DE ESCUCHAR)

La ejecución revierte: quien llama a liberar() es el timelock, no el Governor, así que Ownable responde OwnableUnauthorizedAccount(timelock). Y queda un dueño escondido: con el rol admin, quien desplegó puede darse PROPOSER y agendar operaciones sin votación. Las dos cosas las detectan las pruebas de test/s15/Gobierno.test.js.

B



APROXIMADAMENTE 20 MINUTOS

IDENTIDAD DESCENTRALIZADA

El voto cuadrático, «una persona, un voto», las credenciales: todo necesita saber quién es quién. ¿Puede una cadena pública responder eso sin volverse una base de datos de personas?

UN IDENTIFICADOR QUE NADIE TE PUEDE QUITAR

Hoy tu identidad digital la guardan plataformas: el correo institucional, la cuenta del banco, el «iniciar sesión con...». Si la plataforma cierra tu cuenta, desapareces. Un DID es un identificador que controlas tú con una clave.

```
did : ethr : 0xb9c5714089478a327f09197987f16f9e5d936e8a
esquema    método    identificador propio del método
```

QUÉ HAY DETRÁS

El DID apunta a un «documento DID» con las claves públicas que lo controlan. Quien tiene la clave privada demuestra que es el dueño firmando (Sesión 3).

EL ERROR DE CONCEPTO TÍPICO

«Un DID guarda mis datos en la cadena». No: un DID es un identificador y unas claves. Tus datos no tienen por qué estar en ninguna cadena, y NO deben.

Estándar: W3C Decentralized Identifiers (DIDs) v1.0, Recomendación del 19 de julio de 2022.



EMISOR, TITULAR Y VERIFICADOR



La empresa verifica la firma de la universidad sin llamar a la universidad. Ana decide a quién mostrar su diploma y cuándo. Estándar: W3C Verifiable Credentials Data Model v2.0 (Recomendación, 15 de mayo de 2025).

QUÉ VA DÓNDE	EN LA CADENA	FUERA DE LA CADENA
El diploma con nombre y cédula	NUNCA	En la billetera de Ana
La clave pública de la USB (su DID)	Puede ir	—
«Este diploma fue revocado»	Un registro de revocación, sin datos personales	—



UN NFT QUE NO SE PUEDE VENDER

Un SBT es un token atado a una cuenta: no se transfiere. La idea se popularizó en 2022 (Weyl, Ohlson y Buterin, «Decentralized Society») y tiene un estándar mínimo, ERC-5192, donde `locked(tokenId) = true` bloquea toda transferencia.

PARA QUÉ SIRVE

- Asistencia a un evento, un curso aprobado.
- Pertenencia a una comunidad.
- Base para «una persona, un voto».
- Reputación que no se puede comprar.

LO QUE SALE MAL

- Si pierdes la clave, pierdes la credencial.
- Es público para siempre: todos ven tu historial.
- Puede marcar negativamente («deudor», «sancionado»).
- No se puede borrar si fue un error.

La tercera fila de la derecha es la que nos lleva a la ley colombiana.



INMUTABLE FRENTE A HABEAS DATA

NORMA COLOMBIANA	LO QUE GARANTIZA	CHOQUE CON UNA CADENA PÚBLICA
Constitución, art. 15	Derecho a conocer, actualizar y rectificar la información que se tenga sobre uno.	Lo escrito en la cadena no se rectifica: solo se añade encima.
Ley 1581 de 2012, art. 8	Solicitar la SUPRESIÓN del dato y revocar la autorización.	No existe forma técnica de suprimir un dato de todos los nodos.
Ley 1266 de 2008 (mod. Ley 2157 de 2021)	El dato negativo de crédito tiene permanencia máxima: el doble de la mora, hasta 4 años.	Un SBT de «deudor» seguiría visible después del plazo legal.



DISCUTAN DOS MINUTOS EN PAREJAS

Una cooperativa de Medellín quiere emitir SBT de «buen pagador» y de «mal pagador» para abaratar el crédito. ¿Qué parte es legal? ¿Qué parte es imposible de cumplir en una cadena pública? ¿Qué rediseñarían?

PARCHES HONESTOS, NINGUNO PERFECTO

SALIDA	QUÉ RESUELVE	QUÉ NO RESUELVE
Solo el hash en la cadena (S6)	El dato personal no se publica; se prueba la integridad del documento.	Un hash de un dato adivinable (una cédula) se puede revertir probando.
Credencial fuera de cadena + registro de revocación	El titular guarda sus datos; el emisor puede revocar.	Revocar no es borrar: el historial de revocaciones sigue siendo público.
Divulgación selectiva / pruebas de conocimiento cero	Probar «soy mayor de edad» sin mostrar la fecha de nacimiento.	Complejidad técnica; aún poco adoptado; depende de emisores confiables.

La regla del curso no cambia: en una cadena pública no van datos personales. Ni cifrados, ni «por ahora».



PREGUNTA DE CONTROL

La universidad propone publicar en Sepolia, como SBT, el diploma de cada egresado con su nombre y su promedio. Den dos razones para no hacerlo y una alternativa que conserve la verificación.

RESPUESTA ESPERADA

(1) Nombre y promedio son datos personales: la Ley 1581 da derecho a suprimirlos, y en la cadena no se puede. (2) Queda público para siempre y atado a una cuenta: cualquiera reconstruye el historial del egresado. Alternativa: credencial verificable firmada por la USB que el egresado guarda y muestra cuando quiere; en la cadena, a lo sumo, el hash del documento y un registro de revocación (el patrón de las Sesiones 6 y 11).

C



APROXIMADAMENTE 70 MINUTOS · EN PAREJAS

LABORATORIO 15

Una DAO completa que gobierna una tesorería, con timelock: armarla, probar sus defensas, recorrer el ciclo con tiempo simulado y dejar evidencia en Sepolia. Y un multisig 2 de 3.



CINCO CONTRATOS, TRES ARCHIVOS DE PRUEBAS

ARCHIVO	QUÉ ES	¿LO COMPLETAN?
<code>contracts/s15/TokenVoto.sol</code>	ERC20Votes: saldos, historial y delegación.	No: se lee
<code>contracts/s15/GobiernoUSB.sol</code>	Governor + GovernorTimelockControl.	No: se lee
<code>contracts/s15/TimelockUSB.sol</code>	TimelockController de OpenZeppelin.	No: se lee
<code>contracts/s15/Tesoreria.sol</code>	Fondos cuyo dueño es el timelock.	No: se lee
<code>contracts/s15/MultisigUSB.sol</code>	Multisig mínimo m de n.	SÍ · TODO 1 y 2
<code>test/s15/Gobierno.test.js</code>	11 pruebas de estructura y defensas.	No: deben pasar ya
<code>test/s15/CicloGobierno.test.js</code>	El ciclo completo con tiempo simulado.	SÍ · TODO 1 a 5
<code>test/s15/Multisig.test.js</code>	11 pruebas del multisig.	No: son la meta

Los contratos del Governor casi no tienen lógica propia: son OpenZeppelin bien armado. Lo que se aprende es a MONTARLO y a recorrerlo.



EL ORDEN DEL DESPLIEGUE IMPORTA

LA FUNCIÓN DESPLEGARDAO() DE LAS PRUEBAS

```
const token      = await ethers.deployContract("TokenVoto");
const timelock   = await ethers.deployContract("TimelockUSB",
    [3600, [], [ethers.ZeroAddress], fundador.address]);
const gobierno   = await ethers.deployContract("GobiernoUSB", [token, timelock]);

await timelock.grantRole(await timelock.PROPOSER_ROLE(), gobierno);
await timelock.grantRole(await timelock.CANCELLER_ROLE(), gobierno);
await timelock.renounceRole(await timelock.DEFAULT_ADMIN_ROLE(), fundador.address);

const tesoreria  = await ethers.deployContract("Tesoreria", [timelock]);
```

01 HUEVO Y GALLINA

El timelock necesita al Governor como proponente, y el Governor necesita al timelock: por eso el fundador es admin un momento.

02 RENUNCIAR

Lo último: renunciar al rol admin. Después, nadie tiene llaves maestras.



COMPLETAR CICLOGOBIERNO.TEST.JS

PRUEBA 1: LA FORMA DE LA SOLUCIÓN · TODO 4 Y 5 SON LAS OTRAS DOS PRUEBAS ★

```
await token.connect(ana).delegate(ana.address); await mine(1); // TODO 1a
await gobierno.connect(ana).propose(targets, values, calldatas, descripcion); // 1b
await mine(2); // TODO 2a · Active
await gobierno.connect(ana).castVote(id, VOTO.AFavor); // TODO 2b
await mine(51); // TODO 2c · Succeeded
await gobierno.queue(targets, values, calldatas, hashDescripcion); // 3a · Queued
await time.increase(RETARDO + 1); // TODO 3b · segundos, no bloques
await gobierno.execute(targets, values, calldatas, hashDescripcion); // ya escrito
```

MOMENTO	SALIDA ESPERADA DE NPX HARDHAT TEST TEST/S15/CICLOGOBIERNO.TEST.JS
al empezar	2 passing · 3 failing (las tres marcadas con ★)
al terminar	5 passing · 0 failing: la parte 2 está lista

Cómo saber que terminó: las 5 en verde, y pueden explicar por qué mine() mueve la votación y time.increase() el timelock.



MULTISIGUSB EN LOCAL, SAFE EN SEPOLIA

3A · MULTISIGUSB (RED LOCAL)

- Copiar andamiaje/s15/MultisigUSB.sol sobre contracts/s15/.
- Completar TODO 1 confirmar() y TODO 2 ejecutar().
- Al empezar: 4 passing · 7 failing.
- Al terminar: 11 passing.

3B · SAFE 2 DE 2 EN SEPOLIA

- Dos billeteras del curso con ETH de prueba (una por integrante).
- Crear el Safe en app.safe.global con umbral 2.
- Proponer enviar 0,001 ETH; el otro confirma; ejecutar.
- Evidencia: hash de la ejecución en el explorador.



SI NO HAY ETH DE PRUEBA

La parte 3b es la única que depende de la red. Si el faucet falla, se entrega la 3a con sus 11 pruebas en verde y se documenta el flujo de Safe con capturas del intento. Nunca con dinero real, nunca con la billetera personal.



LA MISMA DAO, DE VERDAD, EN SEPOLIA

```
npx hardhat run scripts/s15/ciclo-gobierno.js # ensayo local: segundos
npx hardhat run scripts/s15/ciclo-gobierno.js --network sepolia # real: unos 15 minutos
```

SALIDA REAL EN LA RED LOCAL (EXTRACTO)

```
[ 0 s] votos antes 0.0 (sin delegar no hay votos)
[ 0 s] delegar      tx 0x2a80... gas 95.622
[ 0 s] proponer     tx 0x0ff9... gas 69.694
[ 0 s] votar        tx 0xde55... gas 83.256
[ 0 s] encolar      tx 0x46bd... gas 145.311
[ 0 s] ejecutar     tx 0x51ed... gas 98.066
[ 0 s] tesoreria    0.002 → 0.001 ETH
```

- En Sepolia una sola cuenta hace todo: es para ver el ciclo en cadena, no una comunidad.
- Espera ~10 min de votación (50 bloques) y 2 min de timelock.
- Usa el keystore: SEPOLIA_RPC_URL y SEPOLIA_PRIVATE_KEY.

Cómo saber que terminó: la última línea muestra la tesorería bajando 0,001 ETH y el estado Executed. Los hashes van al informe.

LO QUE ATASCA A CASI TODOS

SÍNTOMA	CAUSA Y SOLUCIÓN
El voto se emite, pero la propuesta sale Defeated	Nadie delegó, o se delegó después del snapshot. Delegar y mine(1) ANTES de proponer.
execute() revierte con TimelockUnexpectedOperationState	No se esperó el timelock, o se avanzó con mine(). El retardo es en segundos: time.increase.
execute() revierte con OwnableUnauthorizedAccount	La tesorería tiene de dueño al Governor. Debe ser el timelock.
queue() revierte con GovernorNonexistentProposal	El hash de la descripción no coincide: un espacio de más cambia el id.
state() revierte con GovernorNonexistentProposal	Se calculó el id pero nunca se llamó propose().
Error HHE... de configuración al usar --network sepolia	Falta una variable del keystore: npx hardhat keystore set SEPOLIA_RPC_URL.



QUÉ SE ENTREGA Y CÓMO SE CALIFICA

EVIDENCIA	PESO
Gobierno.test.js 11/11 y CicloGobierno.test.js 5/5 en verde (captura con el comando visible)	30 %
MultisigUSB: Multisig.test.js 11/11 en verde	20 %
Evidencia en Sepolia: direcciones y hashes de ciclo-gobierno.js (o del Safe 2 de 2)	25 %
Reflexión: respuestas a las preguntas de la guía, con datos de SU corrida	25 %



DÓNDE ESTÁ TODO

Guía paso a paso: [laboratorios-evm/guias/s15-gobernanza-dao.pdf](#), con preparación, salidas esperadas, errores y rúbrica. Se entrega en el repositorio del equipo antes de la Sesión 16. Política del curso: sin evidencia verificable, el laboratorio no se califica.

A large orange square occupies the left side of the slide. In the center of this square is a large, bold, dark blue letter 'D'.

APROXIMADAMENTE 20 MINUTOS · SE ENTREGA EN LA S17

EL ENSAYO INDIVIDUAL (RA7)

La única pieza estrictamente individual y escrita del curso: pensar en serio una implicación del ecosistema, anclada en Colombia, con postura propia.



1 500 PALABRAS, UNA POSTURA SOSTENIDA

	DETALLE
Qué es	Análisis crítico de UNA implicación —regulatoria, ética, económica o ambiental— aplicada a Colombia.
Extensión	1 500 palabras ($\pm 10\%$), sin contar referencias ni la declaración de IA.
Fuentes	Mínimo 5, leídas; al menos 2 primarias (norma, documento oficial, artículo académico) y 1 colombiana.
Postura	Una tesis propia, sustentada, que reconoce el mejor argumento contrario y le responde.
IA	Permitida y declarada con el formato del documento. No declararla anula el ensayo.
Fechas y peso	Se plantea hoy (S15), se entrega en la S17. 20 % del tercer corte = 8 % de la nota final.

Todo el detalle —rúbrica, formato, temas— en [material/proyecto/ensayo-individual.pdf](#).



POSTURA, NO RESUMEN

UN ENSAYO FLOJO

- Explica otra vez qué es blockchain.
- Lista pros y contras y no decide.
- Cita sin haber leído, o no cita.
- Podría ser sobre cualquier país.

UN BUEN ENSAYO

- Tesis clara en el primer párrafo.
- Fuentes reales, leídas y discutidas.
- Anclado en Colombia: una ley, un caso, un dato.
- Responde al mejor argumento contrario.

LA DECLARACIÓN OBLIGATORIA, AL FINAL DEL ENSAYO

DECLARACIÓN DE USO DE IA

Herramienta y versión: _____ · Fecha(s): _____

Para qué la usé: ☐ buscar fuentes ☐ resumir ☐ corregir estilo ☐ contraargumentos ☐ otro: ____

Qué NO hizo: la tesis, la estructura y la conclusión son mías.

Anexo: las instrucciones (prompts) principales que usé.

POR DÓNDE EMPEZAR

EJE	TEMA SUGERIDO (HAY MÁS EN EL DOCUMENTO)
Ético	Identidad on-chain y habeas data: ¿puede una credencial pública respetar la Ley 1581?
Regulatorio	¿Basta el reporte a la UIAF para proteger a los usuarios de plataformas de intercambio? (S16)
Económico	Remesas hacia Colombia con stablecoins: ¿inclusión real o nuevo riesgo?
Ambiental	¿Tiene sentido la minería de prueba de trabajo con energía hidroeléctrica colombiana?

CRITERIO DE LA RÚBRICA	PESO
Tesis y postura propia sustentada	30 %
Argumentación y contraargumento	25 %
Uso de fuentes (calidad, lectura real, citación)	20 %
Anclaje en Colombia	15 %
Escritura y declaración de IA	10 %



LA PREGUNTA DE LA SEMANA

Elijan el eje de su ensayo y escriban en UNA frase la postura que van a defender. Si no cabe en una frase, aún no la tienen.

PARA LA PRÓXIMA SESIÓN

- Laboratorio 15: las cuatro partes y la reflexión, en el repositorio antes de la S16 (guía PDF).
- Ensayo individual: tesis en una frase y 5 fuentes elegidas esta semana. Entrega en la S17.
- Avance 2 (S16): la dApp integrada con el flujo completo funcionando.
- Traer ETH de prueba en Sepolia: la S16 despliega en Sepolia y en OP Sepolia.



LO QUE SE LLEVAN DE HOY

Gobernar sin jefe no elimina el poder: lo vuelve visible y lo pone en el código.

Ya no es quién manda, es quién tiene los votos, cuándo se cuentan y cuánto hay que esperar. Falta situarlo en el mundo: cuánto escala, cuánto cuesta y qué dice la ley colombiana.

SESIÓN 16 · ESCALABILIDAD, PUENTES Y MARCO REGULATORIO · AVANCE 2